

university of groningen

Performance Engineering: Lecture #14

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-15

Overview



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

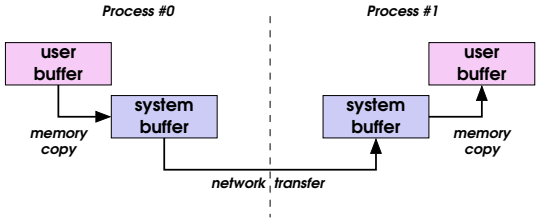
Communication Buffering (1/3)



- MPI communication involves **multiple transactions**
- **Challenges**:
  - Intermediate buffer space required
  - Buffered data must remain unchanged until buffer is completely read

Communication Buffering (2/3)

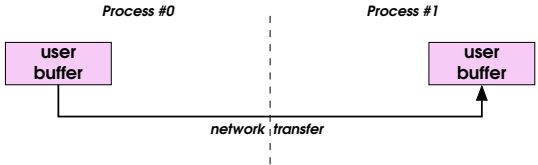
- **Design option #1**: **Intermediate memory buffer**



  - **Advantages**:
    - MPI send completes after fast in-memory copy
  - **Disadvantages**:
    - Need additional space in memory
    - Performing memory copies increases pressure on memory bus

Communication Buffering (3/3)

- **Design option #2**: **Zero-copy communication**



  - **Advantages**:
    - No additional memory space and less memory traffic
  - **Disadvantages**:
    - MPI send only completes once data has been transferred over network

MPI Buffered Mode



- User can explicitly request communication to be buffered (**buffered mode**):

```
1 int MPI_Bsend(
2   const void* buf,        /* Pointer to send buffer */
3   int count,              /* Number of elements     */
4   MPI_Datatype datatype,  /* Data type              */
5   int dest,               /* Destination rank       */
6   int tag,                /* Communication tag      */
7   MPI_Comm comm           /* Communicator           */
8 );
```

- A buffered mode send operation can be started whether or not a matching receive has been posted

MPI Buffered Mode: User-provided Buffer



- A user may specify a buffer to be used for buffering messages sent in buffered mode:

```
1 int MPI_Buffer_attach(
2   void* sysbuf,   /* Pointer to user allocated system buffer */
3   int size        /* Size of buffer in Bytes                 */
4 );
5
6 int MPI_Buffer_detach(
7   void* sysbuf,  /* Returned pointer to system buffer       */
8   int* size      /* Returned size of buffer                 */
9 );
```

Blocking versus Non-Blocking Communication



- **Blocking**
  - Function returns only after completion of the associated operation
  - Examples: `MPI_Send`, `MPI_Recv`
- **Non-blocking**
  - Function may return before the associated operation has completed
  - Examples:
    - Immediate send: `MPI_Isend`
    - Immediate receive: `MPI_Irecv`
  - Resources (e.g. message buffers) passed to function must not be reused until the operation has completed
    - Non-blocking functions return a handle that allow to query status of the initiated operation

MPI Immediate Send and Receive



- Syntax of immediate send operation

```
int MPI_Isend(
  const void* buf,          /* Send buffer                */
  int count,                /* Number of elements         */
  MPI_Datatype datatype,    /* Data type                  */
  int dest,                 /* Destination rank           */
  int tag,                  /* Communication tag          */
  MPI_Comm comm,            /* Communicator               */
  MPI_Request *request      /* Request handle             */
);
```

- Syntax of immediate receive operation

```
int MPI_Irecv(
  const void* buf,          /* Pointer to receive buffer */
  int count,                /* Number of elements        */
  MPI_Datatype datatype,    /* Data type                 */
  int source,               /* Rank of source            */
  int tag,                  /* Message tag               */
  MPI_Comm comm,            /* Communicator              */
  MPI_Request *request      /* Request handle            */
);
```

Waiting for Completion (1/2)



- To wait for a communication identified by a given request handler to complete, the blocking function `MPI_Wait` can be used:

```
1 int MPI_Wait(
2   MPI_Request *request,   /* Request handle (in/out)   */
3   MPI_Status *status      /* Status object (out)       */
4 );
```

  - On return the status object is updated

- `MPI_Test` is available to perform checks without block:

```
1 int MPI_Test(
2   MPI_Request *request,    /* Request handle (in/out) */
3   int *flag,               /* Completion flag (out)   */
4   MPI_Status *status       /* Status object (out)     */
5 );
```

  - The returned `flag` is set to true when operation completed

Waiting for Completion (2/2)



- Use `MPI_Waitall` when waiting for a set of requests to complete:

```
1 int MPI_Waitall(
2   int count,               /* Array size              */
3   MPI_Request request[],   /* Array of requests       */
4   MPI_Status status[]      /* Array of status objects */
5 );
```

MPI Modes Overview (1/2)



- **Standard mode:**
  - A send operation is started whether or not a matching receive has been posted
  - It may complete before a matching receive is posted
- **Synchronous mode:**
  - A send operation can be started whether or not a matching receive has been posted
  - The send operation send will complete successfully only if a matching receive is posted
- **Ready mode:**
  - A send operation may be started only if the matching receive is already posted
- **Buffered mode:**
  - A send operation can be started whether or not a matching receive has been posted
  - It may complete before a matching receive is posted

MPI Modes Overview (2/2)



The following variants of MPI send are available:

Mode	Blocking variant	Non-blocking variant
Standard	`MPI_Send`	`MPI_Isend`
Synchronous	`MPI_SSend`	`MPI_Issend`
Ready	`MPI_RSend`	`MPI_Irsend`
Buffered	`MPI_BSend`	`MPI_Ibsend`

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

MPI One-Sided Communication



- Send-receive communication assumes that both end-points of the communication take action (**two-sided communication**)
- MPI since version 2.0 supports also communication based on a get/put semantics with only one end-point being active (**one-sided communication**)
  - Also called **remote memory access (RMA)**
- Challenges:
  - Need for a common address space
  - Suitable synchronisation to ensure RAW and WAR dependencies to be respected

One-Sided Communication: Basic Operations



- Create a memory window for remote access
- Communicate using, e.g.,
  - `MPI_Put`: Write to remote memory
  - `MPI_Get`: Read from remote memory
- Synchronise using `MPI_Win_fence`

Memory Window Creation



- Specify a window of existing memory that processes exposes to RMA accesses:

```c
int MPI_Win_create(
  void *base,      /* Initial address of window                    */
  MPI_Aint size,   /* Size of window [Bytes]                       */
  int disp_unit,   /* Local unit size for displacements [Bytes]    */
  MPI_Info info,   /* Info object                                  */
  MPI_Comm comm,   /* Communicator                                 */
  MPI_Win *win     /* Window handle (out)                          */
);
```

- Operation is a collective call, all processes must participate
- All address computations are scaled by the `disp_unit`
  - Choose `disp_unit` $= 1$ for no scaling
- The `MPI_Info` object allows to provide optimization hints (not covered here)
  - `MPI_Info` allows to store an unordered set of (key,value) pairs
  - May use `info` $=$ `MPI_INFO_NULL`

Synchronisation



- All RMA calls related to a particular window can be synchronised using

```
int MPI_Win_fence(
  int assert,   /* Program assertion */
  MPI_Win win   /* Window handle     */
);
```

- The application may make assertions, which may allow for optimizations but `assert` $= 0$ is always valid
- `MPI_Win_fence` separates different **access epochs**
  - Epoch starts with an RMA synchronisation call on `win`
  - Next, a number of communication calls may happen
  - Epoch stops with another synchronisation call
- Other synchronisation mechanisms available (not considered here)

MPI_Put (1/2)



- The analogue of a send operation is the following operation that puts data into remote memory:

```
1  int MPI_Put(
2    void *oaddr,              /* Address of origin buffer                    */
3    int ocount,               /* Number of entries in origin buffer          */
4    MPI_Datatype odatatype,   /* Data type of entries in origin buffer       */
5    int trank,                /* Target rank                                 */
6    MPI_Aint tdisp,           /* Displacement from window start to target buffer */
7    int tcount,               /* Number of entries in target buffer          */
8    MPI_Datatype tdatatype,   /* Data type of entries in origin buffer       */
9    MPI_Win win               /* Window handle                               */
10 );
```

- The data is
  - Read from the local memory at address `oaddr`
  - written to the remote memory at address
    `taddr` $=$ `window_base` $+$ `tdisp` $\times$ `disp_unit`

MPI_Put (2/2)



- Need synchronisation to avoid
  - Remote data is overwritten before read of old data (WAR)
  - Remote data is used before new data is written (RAW)

MPI_Get



- To copy data from the target memory to the origin use

```
1  int MPI_Get(
2    void *oaddr,              /* Address of origin buffer                    */
3    int ocount,               /* Number of entries in origin buffer          */
4    MPI_Datatype odatatype,   /* Data type of entries in origin buffer       */
5    int trank,                /* Target rank                                 */
6    MPI_Aint tdisp,           /* Displacement from window start to target buffer */
7    int tcount,               /* Number of entries in target buffer          */
8    MPI_Datatype tdatatype,   /* Data type of entries in origin buffer       */
9    MPI_Win win               /* Window handle                               */
10 );
```

- Need synchronisation to avoid remote data is
  - **Updated** before get has started (RAW)
  - **Not updated** before get completed (WAR)

One-Sided Communication: Ping (1/2)



```
1  int main(int argc ,char *argv[]) {
2    MPI_Init(NULL, NULL);
3
4    int irank;
5    MPI_Comm_rank(MPI_COMM_WORLD, &irank);
6
7    int buf = -1; MPI_Win win;
8    MPI_Win_create(&buf, sizeof(int), sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD, &win);
9
10   int src = 1; int dst = 0;
11   MPI_Win_fence(0, win);
12   if (irank == src) {
13     int x = 100 + irank;
14     MPI_Put(&x, 1, MPI_INT, dst, 0, 1, MPI_INT, win);
15     printf("#%d: have put x=%d\n", irank, x);
16   }
17   MPI_Win_fence(0, win);
18
19   printf("#%d: buf=%d\n", irank, buf);
20
21   MPI_Finalize();
22   return 0;
23 }
```

One-Sided Communication: Ping (2/2)



Question: What is the expected output?

One-Sided Communication: Ping (2/2)



Question: What is the expected output?

When using 4 processes (order is not deterministic):

```
#1: have put x=101
#1: buf=-1
#3: buf=-1
#0: buf=101
#2: buf=-1
```

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. **Collective Communication**
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

MPI Collective Operations: Overview



- MPI defines 9 types of collective operations:
  - Barrier
  - Broadcast
  - Gather
  - All-Gather
  - Scatter
  - All-to-all
  - Reduce
  - All-Reduce
  - Reduce-Scatter
  - Scan
- Collective communication is over all of the processes in particular group identified by a communicator
- Standard meanwhile introduced blocking and non-blocking variants

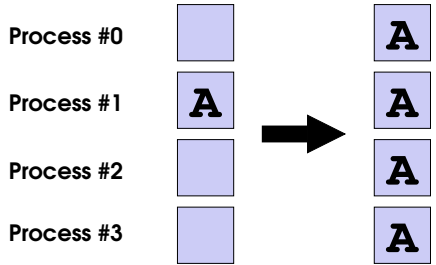
MPI Barrier



- Function blocks the caller until all group members (identified by the communicator `comm`) have called it
  - A deadlock occurs if one group member does not call the function
- Syntax:

  ```
  int MPI_Barrier(MPI_Comm comm)
  ```
- Practical advice
  - In a program with send-receive and other collective communications, an MPI barrier is rarely needed
  - MPI makes no guarantees on how long it will take other processes to leave the barrier
    - MPI barriers are not appropriate for highly accurate time measurements

MPI Broadcast

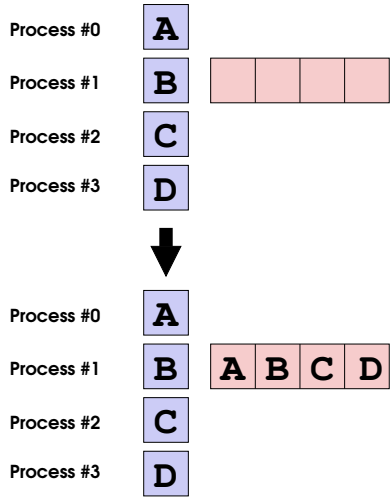


- Function copies data from process identified by `root` and `comm` to all other processes of the group
- Example with 4 processes and `root=1`:
- Syntax:

```
1 int MPI_Bcast(
2   void* buffer,           /* Pointer to output/input buffer */
3   int count,              /* Number of elements             */
4   MPI_Datatype datatype,  /* Data type                      */
5   int root,               /* Rank of root process           */
6   MPI_Comm comm           /* Communicator                   */
7 );
```

MPI Gather (1/2)

- Function collects data on process identified by `root` and `comm` sent by all processes of the group (in rank order)



MPI Gather (2/2)

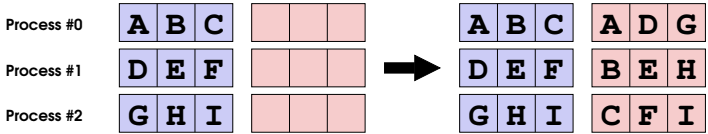


- Syntax:

```
int MPI_Gather(
  const void* sendbuf,    /* Pointer to output buffer  */
  int sendcount,          /* Number of elements        */
  MPI_Datatype sendtype,  /* Data type                 */
  void* recvbuf,          /* Pointer to input buffer   */
  int recvcount,          /* Number of elements        */
  MPI_Datatype recvtype,  /* Data type                 */
  int root,               /* Rank of receiving process */
  MPI_Comm comm           /* Communicator              */
);
```

MPI All-to-all

- Each process sends distinct data to each of the other processes within the group

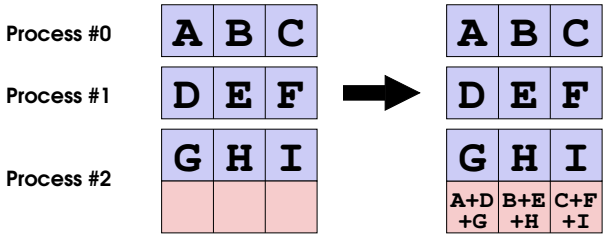


- Syntax:

```
1 int MPI_Alltoall(
2   const void* sendbuf,     /* Pointer to output buffer */
3   int sendcount,           /* Number of elements       */
4   MPI_Datatype sendtype,   /* Data type                */
5   void* recvbuf,           /* Pointer to input buffer  */
6   int recvcount,           /* Number of elements       */
7   MPI_Datatype recvtype,   /* Data type                */
8   MPI_Comm comm            /* Communicator             */
9 );
```

MPI Reduce (1/3)

- Function combines the elements provided in the input buffer of each process in the group, using the operation op, and returns the combined value in the output buffer of the process with rank `root`



MPI Reduce (2/3)



- Syntax:

```
int MPI_Reduce(
 const void* sendbuf,    /* Pointer to output buffer */
 void* recvbuf,          /* Pointer to input buffer */
 int count,              /* Number of elements */
 MPI_Datatype datatype, /* Data type */
 MPI_Op op,              /* Reduction operator */
 int root,               /* Rank of root process */
 MPI_Comm comm           /* Communicator */
);
```

MPI Reduce (3/3)



- MPI's predefined reduction operations:

Name	Meaning
MPI_MAX	maximum
MPI_MIN	minimum
MPI_SUM	sum
MPI_PROD	product
MPI_LAND	logical and
MPI_BAND	bit-wise and

Name	Meaning
MPI_LOR	logical or
MPI_BOR	bit-wise or
MPI_LXOR	logical exclusive or (xor)
MPI_BXOR	bit-wise exclusive or (xor)
MPI_MAXLOC	max value and location
MPI_MINLOC	min value and location

MPI Reduce Example: Global Sum



```c
int main() {
    const int N=1048576;
    int isize, irank;

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &irank);
    MPI_Comm_size(MPI_COMM_WORLD, &isize);

    int n = N / isize;  /* Assume N to be multiple of isize */
    int *v = (int *) calloc(n, sizeof(int));

    for (int i = 0; i < n; i++) v[i] = irank * n + i + 1;

    double s = 0;       /* Local sum */
    for (int i = 0; i < n; i++) s += v[i];

    double gs = -1;     /* Global sum */
    MPI_Reduce(&s, &gs, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (irank == 0) printf("N=%d, gs=%.2f\n", N, gs);

    free(v);
    MPI_Finalize();

    return 0;
}
```

Non-Blocking Collectives



- Collective operations are typically also available in a non-blocking variant
  - Examples
    - `MPI_Ibcast`
    - `MPI_Ireduce`
- These functions return a request handle such that `MPI_Wait` or `MPI_test` can be used to check for completion

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. Final

P2P Communication Optimization: Introduction



- Optimization strategies:
  - Latency hiding
  - Bandwidth maximisation
    - Reduce communication overhead by one-sided communication (not further considered here)
    - Manual data packing
    - Use of MPI's derived data types
  - Optimized mapping to physical network topology

Digression: Software Pipelining



[V.H. Allan et al.; 1995]

- **Software pipelining** = Optimization technique where tasks from different loop iterations are overlapped
- **Goal**: Perform tasks (or instructions) on which other tasks (instructions) depend as early as possible
  - Example: Start send well before data needs to be received

Pipelining: Blocking Communication



- **Assumption**: Routine comprising a routine where in each iteration
  - Data is send to another node
  - Data is received from another node
  - Received data is processed

- **Example algorithm**:

  send $a^{(0)}$;
  for $i = 0$; $i < N - 1$; $i++$ **do**
    send $a^{(i+1)}$;
    receive $a^{(i)}$;
    process $a^{(i)}$;
  end
  receive $a^{(N-1)}$;
  process $a^{(N-1)}$;

Pipelining: Non-blocking Communication



- **Same assumption** but now using non-blocking communication

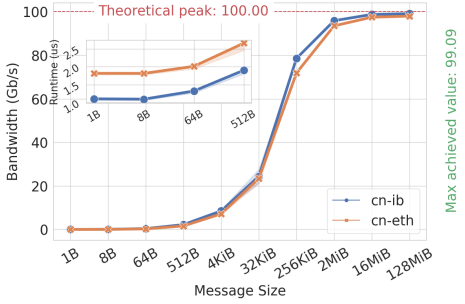
- **Example algorithm**:

$h_0 =$ immediate send $a^{(0)}$;
for $i = 0$; $i < N-1$; $i++$ **do**
  if $i < N-2$ **then**
    $h_{(i+1)\%2} =$ immediate send $a^{(i+1)}$;
  end
  wait $h_{i\%2}$;
  receive $a^{(i)}$;
  process $a^{(i)}$;
end
wait $h_{(N-1)\%2}$;
receive $a^{(N-1)}$;
process $a^{(N-1)}$;

Bandwidth versus Message Size

- The effective bandwidth **depends on the message size**
- Root causes:
  - Limited message rate
  - Relative impact of communication overhead (e.g. message header) higher for small messages

Example: HAICGU cluster



[L. Pichetti et al.; 2024]

Optimization: (Manual) Data Packing



- **Idea**:
  - Copy **scattered** data items **continguously** into a communication buffer
  - Send a set of data in a single message
- **Advantage(s)**:
  - Less and larger messages
- **Disadvantage(s)**:
  - Memory copy operations potentially expensive
  - Data copy cannot easily be overlapped with data processing

Derived Data Types



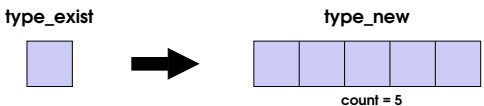
- A **derived data** type (also called general data type) is an opaque object that specifies two things:
  - A sequence of basic data types
  - A sequence of integer (byte) displacements
- Creating derived data types involves the following steps:
  - Definition of derived data types through type constructor functions
  - Commit of the new type
  - Release of all types

Derived Data Types: Contiguous Type



- Constructor allows replication of a data type into contiguous locations:

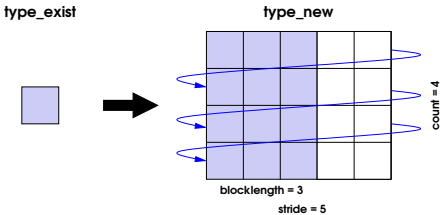
```
1 int MPI_Type_contiguous(
2   int count,                  /* Replication count  */
3   MPI_Datatype type_exist,    /* Existing data type */
4   MPI_Datatype *type_new      /* New data type      */
5 );
```



Derived Data Types: Vector Type



- `MPI_Type_vector` is a more general constructor that allows replication of a data type into locations that consist of equally spaced blocks:




```
1  int MPI_Type_vector(
2    int count,          /* Number of blocks          */
3    int blocklength,    /* Number of elements in
4                           each block                */
5    int stride,         /* Number of elements
6                           between start of
7                           each block                */
8
9    MPI_Datatype type_exist,/* Existing data type */
10   MPI_Datatype *type_new  /* New data type      */
11 );
```

Process Topologies



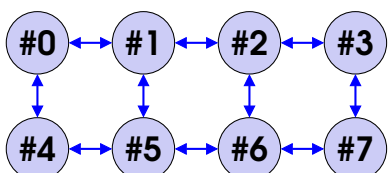
- Communicators can be used to define a **virtual network topology** that defines the neighbourship relations between the processes within the given group
- Benefits of such virtual topologies:
  - Allow to express communication patterns in a more natural way
  - Provide middleware with hints about target communication patterns
    - MPI may use this information to select a better mapping of processes to underlying **physical network topology** (if user allows for rank re-ordering)

Process Topologies: Cartesian (1/2)

- With a Cartesian communicator processes are assumed to be arranged on a one or higher-dimensional mesh or torus

```
1 int MPI_Cart_create(
2   MPI_Comm comm,          /* Existing communicator          */
3   int ndims,              /* Number of dimensions of grid   */
4   const int dims[],       /* Grid size                      */
5   const int periods[],    /* True (false) if grid is (not)
6                              periodic                       */
7   int reorder,            /* Allow to reorder or not        */
8   MPI_Comm *comm_cart     /* New communicator               */
9 );
```

- Example with 8 processes and
  - `ndims` $= 2$
  - `dims` $= [4, 2]$
  - `periods` $= [0, 0]$
  - `reorder` $= 0$



Process Topologies: Cartesian (2/2)



- MPI provides a (convenience) function to compute source and destination to communication with the grid:

```
1 int MPI_Cart_shift(
2   MPI_Comm comm_cart, /* Cartesian communicator                    */
3   int direction,      /* Communication direction                   */
4   int disp,           /* Displacement (sign determins direction)   */
5   int *source,        /* Returned source rank                      */
6   int *dest           /* Returned destination rank                 */
7 );
```

  - The returned `source` and `dest` ranks can be directly used as input for `MPI_Sendrecv`

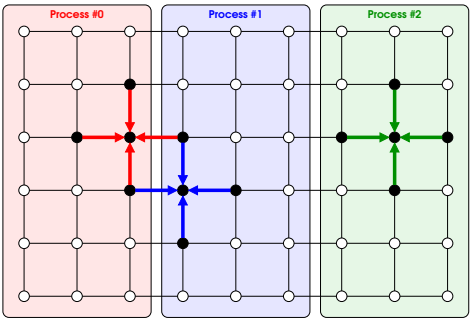
Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. **Case Study: Halo Update (1)**
6. Final

Solving the 2D Poisson in Parallel

- Graphical representation of a parallel version of the discrete 2-dimensional Poisson equation:

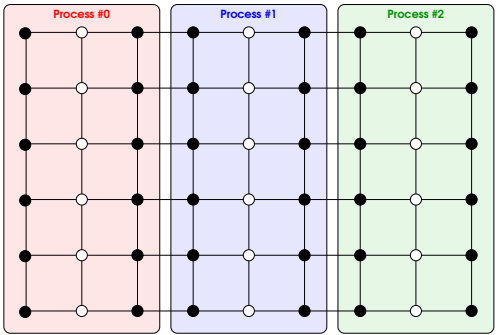


- Local update may require data from remote process

2D Poisson: Halo versus Bulk

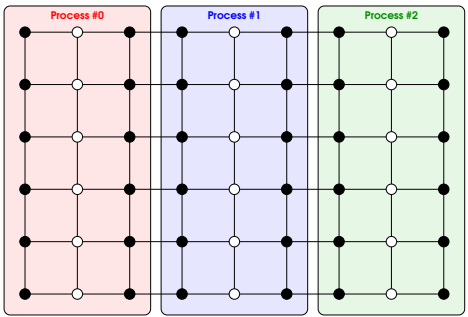
- Classification of grid points:
  - Inner points (**bulk**): No data from other processes needed
  - Boundary points (**halo**): Data from other processes needed

- Parallel update requires **halo exchange**

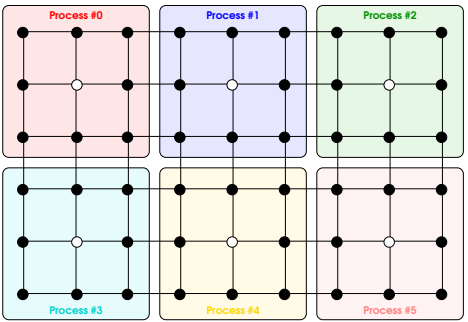


2D Poisson Equation: Data Decomposition

Data decomposition in 1 dimension:



Data decomposition in 2 dimensions:



2D Poisson Equation: Information Exchange



- **Assume** a square lattice of size L^2 being distributed over P processes and use of double-precision numbers
- Update of each site requires 8 Flop:

$$I_{\mathrm{fp}}(L,P) = (8 \cdot L^2/P)\,\mathrm{Flop}$$

- In each iteration, the field v is updated and the local halo needs to be communicated
  - Data decomposition in 1 dimension:

$$I_{\mathrm{net}}^{(1\mathrm{d})}(L,P) = (2 \cdot L \cdot 8)\,\mathrm{B}$$

  - Data decomposition in 2 dimensions:

$$I_{\mathrm{net}}^{(2\mathrm{d})}(L,P) \simeq \left(4 \cdot (L/\sqrt{P} - 1) \cdot 8\right)\,\mathrm{B}$$

2D Poisson Equation: Data Decomposition Analysis



- For fixed P we find

$$\frac{I_{\mathrm{fp}}}{I_{\mathrm{net}}} \propto L$$

  - Increasing L the amount of computation relative to network communication increases
- For fixed L we find

$$\frac{I_{\mathrm{net}}^{(1\mathrm{d})}}{I_{\mathrm{fp}}} \propto P, \qquad \frac{I_{\mathrm{net}}^{(2\mathrm{d})}}{I_{\mathrm{fp}}} \propto \sqrt{P}$$

  - 2-dimensional data decomposition requires less data to be communicated (for sufficiently large L)
  - But: In higher dimensions halo becomes fragmented in memory

Content



1. Non-Blocking Communication
2. One-Sided Communication
3. Collective Communication
4. P2P Communication Optimization
5. Case Study: Halo Update (1)
6. **Final**

Summary Lecture #14



- Finished introduction to MPI: blocking two-sided, one-sided communication, collective communication
- P2P communication optimization
  - Latency hiding
  - Bandwidth maximisation
  - Optimized mapping to physical network topology: data packing, use of MPI derived data types
- Case study on halo update

Questions?



[Wiki Media]